

```
=====
C PROJECT CODE DUMP
Generated on: Thu Mar 19 01:12:48 PM CDT 2026
Directory: /home/eg1316/Engineering-Coursework/EG-2312_Data_Structures_&_Algorithms/exam_1
=====
```

```
=====
FILE: ./code/ADTExceptions.c
=====
```

```
//-----
// Nick Galus
//-----
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#include "ADTExceptions.h"

//-----
void RaiseADTException(const char exception[])
//-----
{
    fprintf(stderr, "\a\nException \"%s\"\n\n", exception);
    system("PAUSE");
    exit( 1 );
}
```

```
=====
FILE: ./code/ADTExceptions.h
=====
```

```
//-----
// Nick Galus
//-----
#ifndef ADTEXCEPTIONS_H
#define ADTEXCEPTIONS_H

// ADT exception definitions
#define MALLOC_ERROR           "malloc() error"
#define DATE_ERROR             "DATE error"
#define STACK_CAPACITY_ERROR   "STACK capacity error"
#define STACK_UNDERFLOW        "STACK underflow"
#define STACK_OVERFLOW         "STACK overflow"
#define STACK_OFFSET_ERROR     "STACK offset error"

// ADT exception-handler prototype
void RaiseADTException(const char exception[]);

#endif
```

```
=====
FILE: ./code/Examination1Part2.c
=====
```

```
//-----
// Nick Galus
//-----
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#include "Stack.h"

//-----
int main()
//-----
{
    void DisplayStack(const STACK *stack);
```

```

int n;

printf("n? ");
while ( scanf("%d",&n) != EOF )
{
    STACK stack;

    ConstructSTACK(&stack,1);
    DisplayStack(&stack);

    for (int i = 1; i <= n; i++)
    {
        PushSTACK2(&stack,i);
        DisplayStack(&stack);
    }

    DestructSTACK(&stack);
    printf("\nn? ");
}
return( 0 );
}

//-----
void DisplayStack(const STACK *stack)
//-----
{
    printf("( %3d,%3d) :",GetSizeSTACK(stack),GetCapacitySTACK(stack));

    if ( GetSizeSTACK(stack) == 0 )
        printf(" (empty)\n");
    else
    {
        for (int offset = GetSizeSTACK(stack)-1; offset >= 0; offset--)
            printf(" %3d",PeekSTACK(stack,offset));
        printf("\n");
    }
}

```

```

=====
FILE: ./code/Stack.c
=====

```

```

// Nick Galus
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

```

```

#include "Stack.h"
#include "ADTExceptions.h"

```

```

//-----
void ConstructSTACK(STACK *stack,const int capacity)
//-----
{
    if ( capacity <= 0 ) RaiseADTException(STACK_CAPACITY_ERROR);

    stack->size = 0;
    stack->capacity = capacity;
    stack->elements = (int *) malloc(sizeof(int)*capacity);

    if ( stack->elements == NULL ) RaiseADTException(MALLOC_ERROR);
}

```

```

//-----
void DestructSTACK(STACK *stack)
//-----
{

```

```

    free(stack->elements);
}

//-----
void PushSTACK(STACK *stack,const int element)
//-----
{
    if ( IsFullSTACK(stack) ) RaiseADTException(STACK_OVERFLOW);

    stack->elements[stack->size] = element;
    ++stack->size;
}

//-----
void PushSTACK2(STACK *stack,const int element)
//-----
{
    if ( IsFullSTACK(stack) )
    {
        int newCapacity = (stack->capacity > 0) ? stack->capacity * 2 : 1;
        int *newElements = (int *) realloc(stack->elements, sizeof(int) * newCapacity);
        if ( newElements == NULL ) RaiseADTException(MALLOC_ERROR);
        stack->elements = newElements;
        stack->capacity = newCapacity;
    }

    stack->elements[stack->size] = element;
    ++stack->size;
}

//-----
void PopSTACK(STACK *stack)
//-----
{
    if ( IsEmptySTACK(stack) ) RaiseADTException(STACK_UNDERFLOW);

    stack->size--;
}

//-----
int PeekSTACK(const STACK *stack,const int offset)
//-----
{
    if ( !((0 <= offset) && (offset <= GetSizeSTACK(stack)-1)) ) RaiseADTException(STACK_OFFSET_ERROR);

    return( stack->elements[stack->size-(offset+1)] );
}

//-----
int GetSizeSTACK(const STACK *stack)
//-----
{
    return( stack->size );
}

//-----
int GetCapacitySTACK(const STACK *stack)
//-----
{
    return( stack->capacity );
}

//-----
bool IsFullSTACK(const STACK *stack)
//-----
{
    return( (stack->size == stack->capacity) ? true : false );
}

```

```

//-----
bool IsEmptySTACK(const STACK *stack)
//-----
{
    return( stack->size == 0 );
}

//-----
void OutputSTACK(const STACK *stack, FILE *OUT)
//-----
{
    fprintf(OUT, "[%d:%d] ", GetSizeSTACK(stack), GetCapacitySTACK(stack));
    if ( IsEmptySTACK(stack) )
        fprintf(OUT, "(empty)");
    else
    {
        int offset;
        int UB = GetSizeSTACK(stack)-1;

        for (offset = 0; offset <= UB; offset++)
            fprintf(OUT, "%d%c", PeekSTACK(stack, offset), ((offset == UB) ? '.' : ' '));
    }
}

//-----
void CopySTACK(STACK *LHS, const STACK *RHS)
//-----
{
    if ( LHS != RHS )
    {
        int offset;

        for (offset = GetSizeSTACK(RHS) - 1; offset >= 0; offset--)
            PushSTACK(LHS, PeekSTACK(RHS, offset));
    }
}

//-----
bool EQSTACK(const STACK *LHS, const STACK *RHS)
//-----
{
    bool r;
    int offset;

    if ( LHS->size != RHS->size ) return( false );

    r = true;
    for (offset = 0; offset <= LHS->size - 1; offset++)
    {
        if ( PeekSTACK(LHS, offset) != PeekSTACK(RHS, offset) )
        {
            r = false;
            break;
        }
    }

    return( r );
}

```

```

=====
FILE: ./code/Stack.h
=====

```

```

//-----
// Nick Galus

```

```
//-----
#ifndef STACK2_H
#define STACK2_H

#include <stdbool.h>

//=====
// Data model definitions
//=====
typedef struct STACK
{
    int size;
    int capacity;
    int *elements;
} STACK;

//=====
// Public member function prototypes
//=====
void ConstructSTACK(STACK *stack, const int capacity);
void DestructSTACK(STACK *stack);
void PushSTACK(STACK *stack, const int element);
void PushSTACK2(STACK *stack, const int element);
void PopSTACK(STACK *stack);
int PeekSTACK(const STACK *stack, const int offset);
int GetSizeSTACK(const STACK *stack);
int GetCapacitySTACK(const STACK *stack);
bool IsFullSTACK(const STACK *stack);
bool IsEmptySTACK(const STACK *stack);
void OutputSTACK(const STACK *stack, FILE *OUT);
void CopySTACK(STACK *LHS, const STACK *RHS);
bool EQSTACK(const STACK *LHS, const STACK *RHS);

//=====
// Private utility member function prototypes
//=====
// (none)

#endif
```

PROGRAM OUTPUT

```
=====
n? 1
( 0, 1): (empty)
( 1, 1): 1

n? 2
( 0, 1): (empty)
( 1, 1): 1
( 2, 2): 1 2

n? 3
( 0, 1): (empty)
( 1, 1): 1
( 2, 2): 1 2
( 3, 4): 1 2 3

n? 4
( 0, 1): (empty)
( 1, 1): 1
( 2, 2): 1 2
( 3, 4): 1 2 3
( 4, 4): 1 2 3 4

n? 16
( 0, 1): (empty)
```

```
( 1, 1): 1
( 2, 2): 1 2
( 3, 4): 1 2 3
( 4, 4): 1 2 3 4
( 5, 8): 1 2 3 4 5
( 6, 8): 1 2 3 4 5 6
( 7, 8): 1 2 3 4 5 6 7
( 8, 8): 1 2 3 4 5 6 7 8
( 9, 16): 1 2 3 4 5 6 7 8 9
(10, 16): 1 2 3 4 5 6 7 8 9 10
(11, 16): 1 2 3 4 5 6 7 8 9 10 11
(12, 16): 1 2 3 4 5 6 7 8 9 10 11 12
(13, 16): 1 2 3 4 5 6 7 8 9 10 11 12 13
(14, 16): 1 2 3 4 5 6 7 8 9 10 11 12 13 14
(15, 16): 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(16, 16): 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

n?